

Action Chunking with Transformers (ACT)

Data Pipeline, Training, and Inference

High-Level Overview

April 14, 2026

- **Action Chunking:** Predicts sequences of actions (chunks) rather than a single step, reducing compounding errors in fine manipulation.
- **Modular Design:** Easily tailored for symmetric or asymmetric setups (e.g., single robotic arm with 2-3 cameras).
- **CVAE Architecture:** Uses a Conditional Variational Autoencoder approach to capture the "style" of human demonstrations.
- **High-Level Flow:** Real-time vision + Proprioception (joint state) + Style variable $\rightarrow$ Action sequence.

- **Why Normalization?** Ensures numerical stability. Prevents motors with large ranges from overpowering fine gripper movements.
- **Vision Data:**
 - Scales pixel values to $[0, 1]$.
 - Normalizes using distribution mean and standard deviation to center color channels and stabilize ResNet layers.
- **Proprioception & Actions:**
 - Standardized using dataset statistics: $x_{norm} = \frac{x_{raw} - \mu}{\sigma}$.
 - Keeps all joint dimensions on a similar scale. Process is reversed before sending commands to hardware.

- **Vision Pipeline (Cameras):**

- Images pass through a **ResNet-18** to extract feature grids.
- Grids are flattened into tokens (e.g., 300 per camera).
- **2D Positional Embeddings** are added so the network remembers *where* features appeared spatially.
- Tokens from all cameras are concatenated into a single visual sequence.

- **Robot State (Proprioception):**

- The lower-dimensional joint state vector maps through a linear layer.
- Produces a **single state token** matching the Transformer's hidden dimension.

The CVAE Encoder (Capturing Style)

- **Goal during training:** Learn the nuance of a human demonstration before the main Transformer handles the scene.
- **Inputs to the Style Encoder:**
 - Current physical state (no images, to save computing power).
 - The exact **future actions** taken by the human.
- **The [CLS] Token:** A designated "dummy" token that interacts bidirectionally with the whole sequence. It acts as a final summary bucket, discarding individual sequences to extract a single **Style Vector**.

- Instead of a static tensor, the style vector represents a **Gaussian distribution**.
- The network predicts the statistical center (Mean μ) and uncertainty (Variance σ^2).
- **The Reparameterization Trick:** To allow training (backpropagation), we sample the style z using random noise: $z = \mu + \sigma \cdot \epsilon$
- This z captures mathematical stochasticity and specific multi-modal aspects of the demonstration block.

The Policy Transformer (Action Generation)

- **Policy Encoder:** Combines vision tokens, state token, and the sampled style token (z).
 - Employs self-attention so these modalities can "talk" and produce a unified memory of the environment and intended style.
- **Policy Decoder:** Queries the Encoder's memory using fixed position blocks.
 - Generates a purely mathematical "sentence" representing the future path.
 - A final linear layer crushes these high-dimensional tokens back directly into physical motor instructions (the Action Chunk).

The Dual Loss Objective

The network improves by minimizing two mathematical penalties simultaneously:

- ① **L1 Reconstruction Loss:** Measures how closely the predicted chunk matches the human expert's ground-truth chunk.
 - Uses L1 over Mean Squared Error (MSE) because physical robotics requires tight adherence to precise, fine-grained details without blurring.
- ② **KL Divergence:** A penalty restricting how the "style" variable (z) behaves in multi-modal situations.

The Multi-Modal Averaging Problem

- **The Problem:** Humans demonstrate tasks with natural variation (e.g., grabbing a cup by the handle or by the top).
- Without a style variable, a basic network tries to mathematically average these conflicting demonstrations, resulting in poor execution (crashing into the cup).
- **The Solution (z):** The style variable z gives the model an excuse to safely separate these approaches. Positive z = Grab the handle; Negative z = Grab the top.

Why the KL Divergence Prior?

- We apply a massive penalty to force all natural style variations to cluster tightly around the mathematical origin $\mathcal{N}(0, I)$.
- By penalizing large values, the origin ($z = 0$) becomes the **Canonical Baseline**—the absolute safest standard way to complete the task.
- It solves the issue for inference: when the human expert isn't there to provide a demonstration, setting $z = 0$ cleanly breaks ties and prevents erratic behavior.

Update of Weights (Optimization Architecture)

- The entire CVAE and Policy networks are optimized simultaneously. Gradients flow backwards directly from the Decoder outputs to the Style Encoder mapping.
- **AdamW Optimizer:** Preferred because it smoothly handles Transformer topologies.
 - **Momentum:** Accelerates training in consistent downward slopes.
 - **Adaptive Learning:** Individually scales learning rates up or down per weight, stabilizing deep internal layers.
 - **Weight Decay:** A regularization technique that stops any single weight from ballooning, combatting overfitting to noise.

The Simplified Inference Pass

- In deployment, there is no human future data to interpret. The **Style Encoder** is **disabled and discarded**.
- **Hardcoding z** : We forcefully set $z = \vec{0}$, dropping the robot directly onto its safest, most robust core strategy.
- **Fast Loop**: The active pipeline distills down to merely reading cameras and proprioception, passing them to the Policy Encoder with $z = 0$, and executing the chunk.

The Overlapping Chunk Problem

- A single prediction provides a 100-step future action sequence.
- Playing 100 steps blindly to the end causes highly unreactive robotic motion—the robot wouldn't notice if something shifted.
- **High Frequency Queries:** ACT re-runs inference every few milliseconds.
- **The Conflict:** Because of constant replanning, the framework ends up with a queue of *overlapping predictions competing for the exact same physical motor execution* at any given millisecond.

- Overlapping chunk conflicts are solved using **Temporal Ensembling**.
- Instead of letting the newest chunk override older ones, they are mathematically blended together via **Exponential Weighting**.
- **Oldest Predictions (High Weights)**: Provide a stable, committed trajectory core over time.
- **Newest Predictions (Low Weights)**: Act as subtle "corrective nudges" accommodating immediate visual changes.
- **Result**: Smooth, continuous, remarkably reactive long-horizon policies.

Questions?